

Plataforma de Delivery para Comércio de Bairro

*Oito serviços, um gateway, e um sistema inteiro
construído em volta de uma pergunta: com
quem está o dinheiro no fim do expediente.*

Versão 2.0 — 24 de agosto de 2026

Estado do sistema marco o concluído · esqueleto, build e infraestrutura montados

Decisões registradas vinte e uma ADRs aceitas, duas sem objeto, uma pendente

Regras de domínio seis documentos, setenta e três invariantes numeradas

Substitui *Arquitetura-Plataforma-Delivery.pdf* (v1.0, 16/08/2026) e incorpora a revisão de escopo v1.1 e as emendas que vinte e uma decisões produziram desde então.

Onde este documento e o repositório divergirem, o repositório está certo: `docs/dominio/` tem a regra vigente, `docs/architecture/decisions/` tem o porquê.

Sumário

- 1** Sobre este documento
o que ele substitui e o que não substitui
- 2** O produto em uma página
o problema, quem compra, o que vende
- 3** As seis premissas
o que determina o desenho, e o que acontece se cair
- 4** Contexto
quem usa, quem é fornecedor
- 5** Decomposição em serviços
os oito, o que saiu, e por quê
- 6** As quatro regras estruturais
banco, anéis, monorepo, gateway
- 7** O modelo de dinheiro
o núcleo do sistema
- 8** O ciclo do pedido
doze estados, vinte e cinco transições
- 9** A saga e o pivô
onde a transação deixa de ser local
- 10** Autorização contextual
permissão pertence ao vínculo
- 11** O fechamento de expediente
a funcionalidade que vende o produto
- 12** Catálogo, entrega e conversa
os outros três domínios
- 13** Eventos e contratos
como os serviços conversam
- 14** Segurança
autenticação, webhooks, segredos
- 15** Privacidade e LGPD
base legal, retenção, exclusão
- 16** As vinte e uma decisões
índice das ADRs
- 17** Marcos de entrega
onde existe produto vendável
- 18** O que ainda não existe
a seção mais honesta deste documento
- 19** Glossário

Sobre este documento

Este documento descreve o sistema como ele deve estar funcionando agora. Não é um plano, não é uma proposta, e não narra a história de como se chegou aqui — para isso existem as ADRs.

A versão 1.0 da arquitetura foi escrita para um marketplace de delivery. Um adendo crítico mostrou que a premissa estava errada: o cliente do produto é o **comerciante**, não o consumidor; a plataforma na maioria das vezes **não custodia o dinheiro**; o entregador **pertence ao estabelecimento**; e o pedido nasce numa **conversa de WhatsApp**. A revisão v1.1 aceitou o diagnóstico. As vinte e uma decisões registradas desde então são as consequências dessa correção, e este documento é o primeiro em que elas aparecem juntas.

1.1 A hierarquia das fontes

Quatro tipos de documento convivem no repositório, e a distinção entre eles é o que impede que um contradiga o outro em silêncio.

ONDE	RESPONDE	MUDA
docs/PRD.md	o que o produto é, para quem, e o que está fora	quando o escopo muda
docs/dominio/	qual é a regra hoje — agregados, invariantes, tabelas de transição, fórmulas	sempre que a regra muda
docs/architecture/decisions/	por que se decidiu assim, com alternativas rejeitadas e custos aceitos	raramente; emenda com data
docs/operacao/	como se executa, sob pressão	quando o procedimento muda
este PDF	o sistema inteiro numa leitura só	por versão; não se reescreve, se republica

Quando dois divergirem, **a divergência é o defeito** — corrige-se na mesma alteração. E quando este PDF divergir do repositório, o repositório vence: ele é o que a pessoa abre para escrever código.

UMA CONVENÇÃO QUE VALE EXPLICAR

Uma ADR aceita não é reescrita por gosto. Ganha um bloco de emenda com data, ou é revogada. Foi assim com as decisões 009, 010 e 014 na revisão v1.1; não foi assim com a 019, que ficou marcada como "aceita" por três dias enquanto já estava revogada em outro documento. É essa falha que a convenção existe para não repetir.

1.2 O que este documento não é

Não é parecer jurídico — a seção sobre LGPD é engenharia com justificativa, e precisa de revisão por advogado antes do primeiro cliente real. Não é manual de operação — não há sistema operando. E não é uma lista de conquistas: a seção 18 é tão longa quanto as outras de propósito, porque um documento de arquitetura que só descreve o que funciona é propaganda.

1.3 Por onde começar

Quem tem quinze minutos lê as seções 3, 7 e 9 — as premissas, o modelo de dinheiro e a saga. É o suficiente para entender por que o sistema é como é. Quem vai escrever código lê a seção 8 inteira, incluindo a tabela de transições, e depois abre `docs/ dominio/`. Quem vai revisar decisões lê a seção 16 e vai direto às ADRs 004, 002, 021 e 009, nessa ordem: são as quatro que sustentam quase todas as outras.

UMA NOTA SOBRE NÚMEROS

Toda invariante citada aqui tem identificador — `I6`, `J9`, `M15`, `C11`, `N3`, `V5` — e o identificador é o mesmo do documento de domínio correspondente. A letra diz o agregado: **I** de pedido, **J** de jornada, **M** de estabelecimento, **C** de catálogo, **N** de entrega, **V** de conversa. Nenhuma delas é preferência de estilo: quebrar qualquer uma é defeito.

O produto em uma página

Uma pizzaria de bairro recebe pedidos pelo WhatsApp, anota no caderno, calcula o troco de cabeça e, às onze da noite, tenta descobrir quanto vendeu e quanto o motoboy tem no bolso. Erra. Discute. Aceita a diferença e vai dormir.

De todas as dores desse fluxo, uma se repete todo dia, envolve dinheiro e gera conflito entre pessoas: **não saber, ao fim do expediente, quanto entrou, por qual método, e com quem está o dinheiro.** É essa que o produto ataca primeiro. Cardápio, pedido e entrega existem para produzir o dado que fecha o caixa.

Seu WhatsApp vira sistema. Seu dia fecha sozinho.

PRD §4 — proposta de valor

2.1 O que o diferencia, tecnicamente

Três características do negócio produzem quase toda a arquitetura, e nenhuma delas é sobre volume ou escala:

CARACTERÍSTICA DO NEGÓCIO	CONSEQUÊNCIA TÉCNICA
Assinatura, não comissão	Não existe campo de percentual sobre venda em lugar nenhum do modelo. O serviceFee foi removido da decomposição do total, e a remoção é deliberada: campo disponível é campo que um dia alguém preenche
A plataforma não custodia o dinheiro	O pagamento não é passo de saga. O que existe é registro de liquidação : método efetivo, valor efetivo e responsável pela custódia, gravados dentro do agregado do pedido
O entregador pertence à loja	Não há leilão, oferta, aceite nem remuneração por corrida. A remuneração vive no vínculo entre entregador e estabelecimento, e é apurada por jornada , não por entrega

2.2 O sinal de que o produto pegou

Não é volume de pedidos. É **o comerciante fechar o expediente pelo sistema por cinco dias seguidos.** Enquanto ele conferir no papel em paralelo, o produto ainda não substituiu nada — e a arquitetura inteira é desenhada para chegar a esse número com um caminho que duas pessoas consigam conferir juntas.

SEÇÃO 3

As seis premissas

Estas premissas não são contexto: são o que determina o desenho. Se alguma deixar de valer, a coluna da direita diz o que volta a ser necessário — e quase sempre é muito.

#	PREMISSA	SE DEIXAR DE VALER
P1	A plataforma não custodia o valor na maioria das transações	Volta a ser necessário adquirente, captura e estorno como caminho principal — e o pivô da saga volta para o pagamento
P2	O entregador pertence ao estabelecimento	Voltam oferta competitiva, aceite, timeout de oferta e remuneração por corrida
P3	O comerciante é o cliente; o consumidor é usuário	Voltam descoberta, busca e navegação entre lojas — e com elas o pedido multiloja
P4	O pedido nasce predominantemente em conversa	O canal deixa de ser fundação e vira integração acessória
P5	A taxa de entrega é por área nomeada , não por distância	Voltam geocodificação, motor de rotas e tipos espaciais
P6	A maioria dos produtos opera em disponibilidade qualitativa , não em saldo contado	Controle quantitativo volta ao caminho principal, com reserva e perna compensável na saga

A mais estrutural é a **P3**. É dela que decorre a decisão de que um pedido pertence a exatamente um estabelecimento, e é essa decisão que sustenta outras cinco: de qual tabela sai a taxa, qual entregador leva, em qual jornada a liquidação entra, contra qual vínculo a permissão é verificada, qual identificador vai no caminho da URL. Cada uma dessas perguntas seria uma decisão nova, com decomposição própria, se um pedido pudesse atravessar duas lojas.

O QUE SE PAGA POR ISSO

"Pizza de um lugar e sobremesa de outro" é impossível no mesmo carrinho, e quem quiser duas lojas faz dois pedidos e paga duas taxas de entrega. É honesto e é o custo real de duas entregas — mas é pior que a experiência de um marketplace, e vale saber disso em vez de descobrir por reclamação.

SEÇÃO 4

Contexto

Três tipos de pessoa usam o sistema e três fornecedores externos o alimentam. Nenhum dos três fornecedores é fundação: cada um vive atrás de uma porta, e a queda de qualquer um deles degrada o produto sem derrubá-lo.

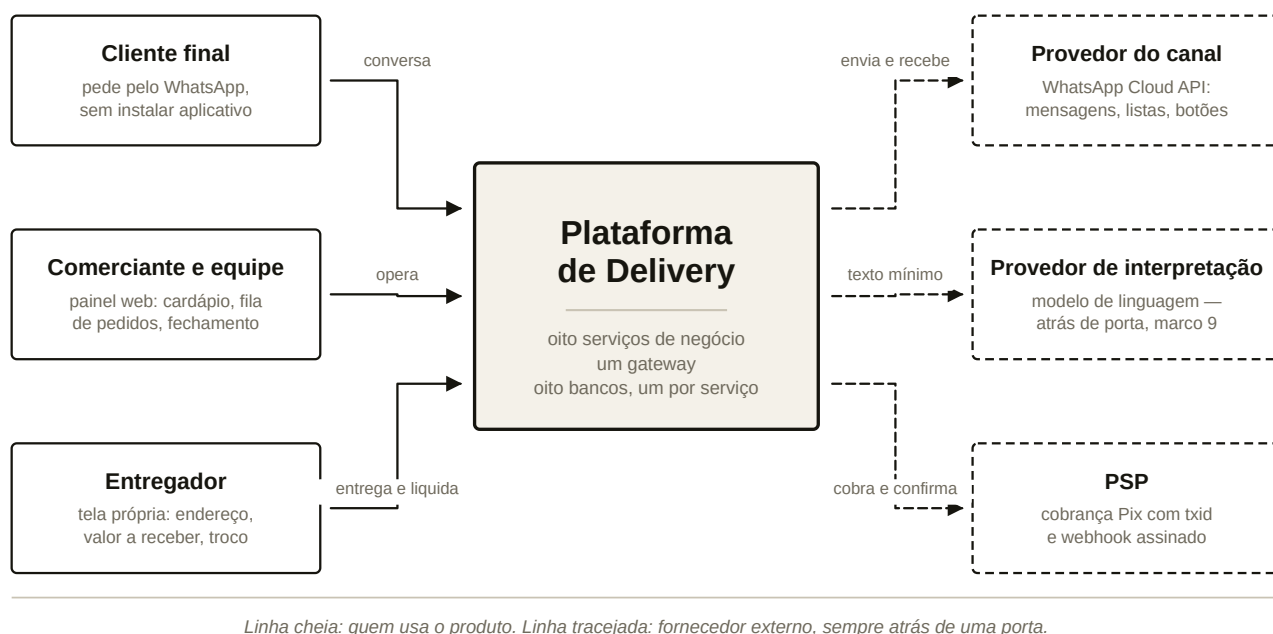


FIGURA 1 — Contexto do sistema. O consumidor nunca instala aplicativo: ele conversa no número da loja. O comerciante opera pelo celular. O entregador tem tela própria, derivada do pedido e sem nenhum campo editável.

4.1 Por que a interpretação automática é camada, não fundação

O canal determinístico — menu numerado, listas interativas, botões — atende o pedido recorrente **de ponta a ponta**, e continua disponível para sempre. A interpretação por modelo de linguagem entra dois marcos depois, e quando a interpretação falha o cliente cai no menu, que funciona.

Quem constrói a inteligência primeiro e o menu depois inverte isso, e o menu vira um caminho pela metade que ninguém testa. Há também um motivo financeiro: a partir de outubro de 2026 cada mensagem enviada pela loja tem preço, e um laço de conversa — interpretação ruim que gera vinte mensagens de esclarecimento — deixa de ser um problema de qualidade e vira risco de margem.

A conversa entende. O sistema decide.

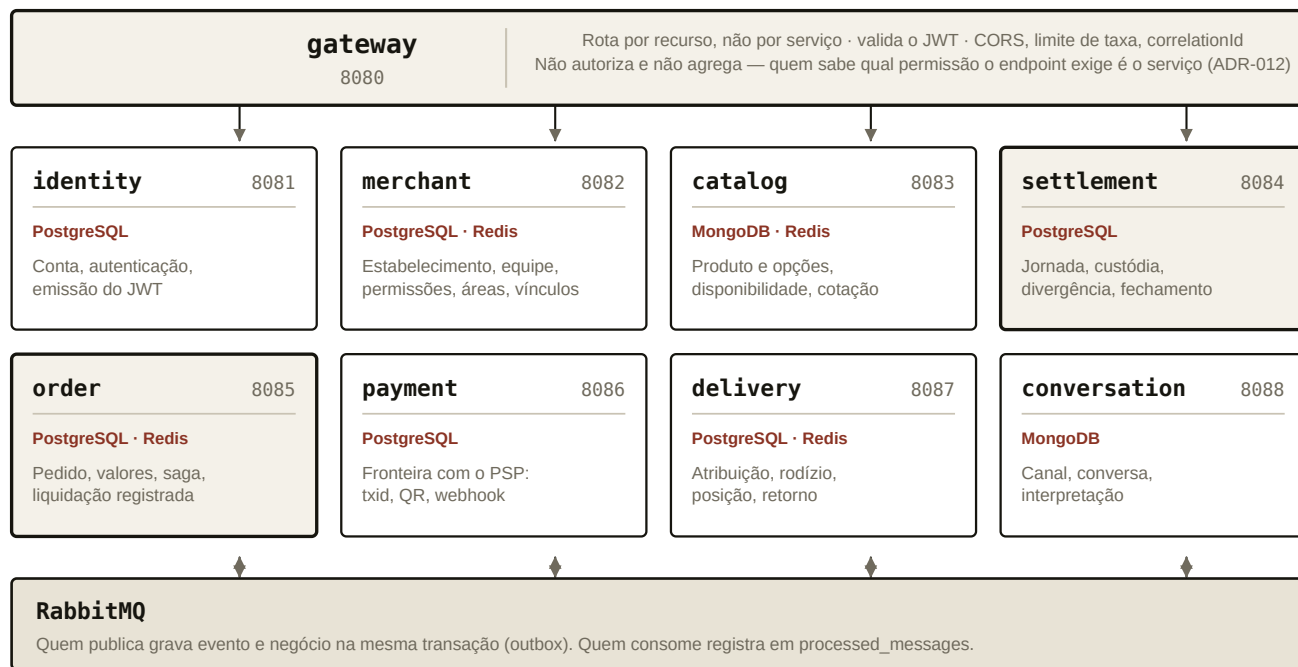
docs/dominio/conversa.md — a regra que organiza o serviço inteiro

A porta de interpretação devolve **identificadores**, nunca valores. E isso não é recomendação: o tipo de retorno dela não tem campo monetário.

SEÇÃO 5

Decomposição em serviços

Oito serviços de negócio e um gateway, cada um com processo, banco, migrations, testes, contêiner, contrato e pipeline próprios.



Fora do MVP: inventory adiado ao marco 10 · geolocation ao marco 11 · notification absorvido pelo conversation, e não volta (ADR-021)
Seis bancos PostgreSQL e dois MongoDB. Nenhum serviço acessa o banco de outro: integração é por API ou evento (ADR-002).

FIGURA 2 — Os oito serviços, suas portas e suas persistências. Destacados, os dois que carregam o núcleo do produto: **order**, dono do registro de liquidação, e **settlement**, dono do fechamento de expediente.

5.1 O que é de cada um

SERVIÇO	PORTA	O QUE É DELE
identity	8081	Conta, autenticação e emissão do JWT. Único serviço que emite; todos os outros validam pela chave pública
merchant	8082	Estabelecimento, equipe, permissões, áreas e taxas, vínculo do entregador. É o serviço do qual todos os outros dependem para autorizar
catalog	8083	Produto e opções, disponibilidade qualitativa, cotação. É o dono do preço — nenhum outro serviço calcula quanto custa alguma coisa
settlement	8084	Jornada do entregador, custódia, divergência de caixa, extrato e fechamento imutável
order	8085	Pedido, valores, máquina de estados, saga e registro de liquidação
payment	8086	Fronteira com o provedor de pagamento: cobrança Pix com txid, QR dinâmico, webhook assinado. Não é livro-caixa

SERVIÇO	PORTA	O QUE É DELE
delivery	8087	Atribuição, rodízio, posição do entregador, retorno registrado
conversation	8088	Canal, conversa, interpretação. Absorveu a notificação ao cliente

5.2 O que saiu, e por quê

O esqueleto foi criado sob a v1.0, com nove serviços desenhados para um marketplace. Durante três dias o repositório teve documentação v1.1 e estrutura v1.0 — e isso já custou uma decisão errada: um agente adicionou configuração de mensageria ao `identity-service` lendo a coluna "publica eventos" de um README que descrevia uma arquitetura superada.

A LIÇÃO QUE FICOU REGISTRADA

Estrutura que contradiz a documentação não é dívida cosmética. É uma fonte de verdade concorrente — e ela vence, porque o código é o que a pessoa abre.

SERVIÇO	DESTINO	VOLTA?
inventory	Adiado — marco 10	Sim, com o controle quantitativo. O que precisava sobreviver sobreviveu: <code>stockControlledSnapshot</code> continua congelado no item, e o estado <code>AGUARDANDO_ESTOQUE</code> continua na máquina, inalcançável e com teste que prova a inalcançabilidade
geolocation	Adiado — marco 11	Sim, se houver operação que justifique. A taxa é por bairro (P5), então não há distância a calcular
notification	Absorvido pelo conversation	Não. O canal com o cliente é a conversa; o alerta do operador é o painel, por consulta periódica; o entregador é avisado pelo operador. Sobram recibos por e-mail — um adaptador de porta, não um serviço

Apagar código que compilava e testava verde é desconfortável, e é o certo. O custo de manter estrutura morta é pago **todo dia** — em build, em leitura e em decisão errada; o custo de reintroduzir é pago uma vez.

5.3 Como se trabalha

Ninguém sobe os nove aplicativos. Sobe-se a infraestrutura e roda-se pelo IDE apenas o serviço em que se está mexendo.

```
docker compose --profile core up -d      # bancos, brokers, storage
./gradlew :services:order-service:bootRun # só o que interessa agora
```

PERFIL DO COMPOSE	SOBE
core	PostgreSQL, MongoDB, Redis, RabbitMQ, MinIO
services	gateway e os oito microsserviços
observability	Prometheus, Grafana, Loki, Tempo
full	tudo — e não se trabalha assim

As quatro regras estruturais

Quatro regras separam este sistema de um monólito com pastas. Três delas são verificadas por build; a quarta ainda não é, e isso está escrito na seção 18.

6.1 Um banco por serviço, sem exceção

O padrão acidental de todo sistema que se divide em serviços é manter um banco só com vários esquemas. Ninguém decide isso: é o que sobra quando não se decide. E funciona, por um tempo — até o dia em que um relatório precisa juntar pedido com cardápio e alguém escreve um `JOIN` entre esquemas.

O `JOIN` entre fronteiras é o momento exato em que a arquitetura morre, e ele sempre parece razoável quando é escrito.

ADR-002

Seis bancos PostgreSQL e dois MongoDB. Integração é por **API** (síncrona, com timeout) ou **evento** (assíncrono, com outbox e idempotência). Em desenvolvimento roda-se uma instância do PostgreSQL com seis bancos lógicos — o isolamento é lógico, e é suficiente, porque o que se está protegendo é a fronteira de modelo, não a de máquina.

Vale dizer em voz alta o que esta decisão custa, porque quem lê a saga e o outbox sem ela acha que é sofisticação gratuita:

CONSEQUÊNCIA	ONDE VIVE
Sem transação distribuída → saga com pivô e compensação	Seção 9
Sem gravar dado e publicar evento atomicamente → outbox	Seção 13
Mensagem se repete → processed_messages	Seção 13
Sem JOIN entre serviços → duplicação deliberada	Seção 11
Permissão em outro serviço → porta com cache e fail-closed	Seção 10

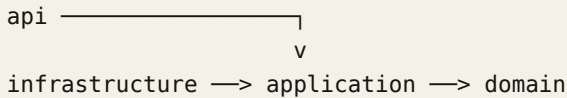
A duplicação é intencional e tem direção. O `settlement` guarda sua própria cópia da liquidação, alimentada por evento, e nunca lê a tabela do `order`. Uma cópia com um dono e um sentido é dado replicado; duas cópias que se escrevem mutuamente é corrupção esperando data.

METADE DA COMPLEXIDADE VEM DAQUI

Saga, outbox, idempotência, duplicação — nada disso existiria com um banco só. Quem discordar desta decisão está discordando de todo esse conjunto, e é bom que a conta esteja num lugar só.

6.2 Hexagonal por anéis, verificado por teste

Dentro de cada serviço, a divisão é por **anel de dependência**, não por camada técnica.



- `domain` não conhece Spring, JPA nem MongoDB. Regra pura; interfaces de repositório ficam aqui.
- `application` orquestra casos de uso e depende de portas de saída.
- `infrastructure` implementa persistência, mensageria, clientes e segurança.
- `api` traduz HTTP em comando.
- Entidade JPA vive em `infrastructure/persistence/entity`, **nunca** em `domain`. A duplicação entre modelo de domínio e entidade é intencional.

Isso é verificado, não combinado. Cada serviço tem um teste de ArchUnit que quebra o build se `domain` importar framework, se uma classe de domínio tiver anotação de persistência, ou se a direção das dependências for invertida. Em revisão manual, a arquitetura erode em duas semanas.

6.3 Monorepo com fronteira explícita

Um repositório, com fronteiras internas. Para uma pessoa, a diferença de sobrecarga entre um e oito repositórios é a diferença entre trabalhar e administrar — e uma renomeação de evento que toca produtor, consumidor e contrato cabe em um commit, validado de uma vez pelo CI.

O que torna isto microsserviços e não um monólito com pastas são três coisas, e nenhuma delas é o diretório: **processo próprio** (cada serviço tem `bootJar`, `Dockerfile` e porta), **banco próprio**, e **pipeline próprio com filtro de caminho** — alterar o `catalog-service` não dispara o CI do `order-service`.

Nenhum módulo de serviço declara outro módulo de serviço como dependência.

ADR-001 — a regra que impede o monorepo de virar monólito

É a fronteira de verdade. Num monorepo, o `order-service` importar uma classe do `merchant-service` é duas linhas de configuração — resolve o problema de hoje e destrói a independência que justifica a arquitetura inteira.

E ESTA É A REGRA QUE AINDA NÃO É VERIFICADA

O teste de ArchUnit checa camadas *dentro* de um serviço; nada checa dependência *entre* serviços. Precisa de uma verificação no build que falhe se `:services:X` declarar `:services:Y`. Requisito do marco 1 — até existir, esta decisão depende de ninguém errar.

6.4 O gateway roteia por recurso e autentica sem autorizar

As rotas públicas não mencionam serviço nenhum. O cliente não sabe que existe `order-service`: sabe que existe pedido, e que pedido pertence a um estabelecimento.

```
/api/v1/auth/**          → identity
/api/v1/me/**            → identity

/api/v1/merchants/{merchantId}/settings/** → merchant
/api/v1/merchants/{merchantId}/team/**    → merchant
/api/v1/merchants/{merchantId}/areas/**   → merchant
/api/v1/merchants/{merchantId}/couriers/** → merchant
/api/v1/merchants/{merchantId}/catalog/** → catalog
/api/v1/merchants/{merchantId}/orders/**  → order
/api/v1/merchants/{merchantId}/deliveries/** → delivery
/api/v1/merchants/{merchantId}/shifts/**  → settlement
/api/v1/merchants/{merchantId}/payments/** → payment

/api/v1/couriers/me/**    → delivery

/api/v1/webhooks/psp/**   → payment      (público, assinado)
/api/v1/webhooks/channel/** → conversation (público, assinado)
```

Este repositório já produziu a prova de que a decomposição interna não pode vazar para a URL: uma revisão de escopo apagou três serviços e criou dois. Se aquelas rotas tivessem cliente, teria sido quebra de contrato público — e a refatoração, que foi a coisa mais saudável que aconteceu no projeto, teria custado uma migração de API.

O `merchantId` fica sempre no segundo e terceiro segmentos. Não é estética: há **um** lugar de onde extraí-lo, e a regra de que identificador de URL não é confiável fica verificável do mesmo jeito em todo endpoint.

RESPONSABILIDADE	GATEWAY	SERVIÇO
Validar assinatura e validade do JWT	sim	sim, de novo
Rejeitar requisição sem token onde token é exigido	sim	—
Saber qual permissão o endpoint exige	não	sim
Confrontar <code>merchantId</code> com o vínculo do usuário	não	sim
CORS, limite de taxa, <code>correlationId</code>	sim	—

O gateway **não pode** autorizar porque não sabe que uma escrita em pedidos exige `ALTERAR_STATUS` e uma leitura exige `VER_PEDIDO`. Colocar esse mapa na borda é duplicar regra de domínio onde ela envelhece sem ninguém ver. E o serviço revalida o token — não é redundância desperdiçada: é o que impede que alcançar a rede interna equivalha a estar autorizado.

O gateway também **não agrega**. Uma requisição, um serviço. Agregação transforma o roteador em quem conhece todos os domínios, e é assim que se constrói um monólito distribuído com um proxy no meio.

SEÇÃO 7

O modelo de dinheiro

O sistema precisa cobrar o valor certo, provar depois qual foi, e sobreviver ao fato de que na maioria dos pedidos **o dinheiro nunca passa pela plataforma**: o cliente paga ao entregador, na porta de casa, em dinheiro, cartão ou Pix.

Isso muda o problema. Não basta decompor o valor cobrado — é preciso separar três coisas que o desenho ingênuo funde num campo só.

O QUE É	QUANDO EXISTE	QUEM SABE
O que o pedido vale	no fechamento do pedido	o sistema, a partir do catálogo
O que o cliente disse que pagaria	no fechamento do pedido	o cliente
O que de fato foi pago, como e a quem	na entrega, ou nunca	o entregador

7.1 Decomposição do valor

itemsSubtotal	Σ dos subtotais dos itens, cada um já arredondado
+ deliveryFee	taxa da área nomeada, congelada
+ tip	gorjeta, campo próprio e explícito
- discount	desconto concedido
= total	valor devido pelo pedido, imutável após o fechamento

Quatro componentes, e não cinco. O `serviceFee` existia para comissão sobre a venda, que a premissa P3 e o modelo por assinatura eliminaram. Um campo zerado para uma cobrança que o produto se comprometeu publicamente a não fazer não é preparação — é convite. Um dia alguém o preenche.

O tipo monetário é um *value object* imutável com `BigDecimal` de escala 2 e código de moeda, com arredondamento explícito. `double` e `float` são proibidos em toda a cadeia. Arredonda-se ao formar **cada componente**; o total é a soma de componentes já arredondados, nunca um recálculo independente — é o que evita a soma exibida na tela não bater com o valor cobrado na porta.

7.2 Método declarado e método liquidado

Declarado ≠ liquidado é o caso normal, não a exceção.

ADR-009 · invariante 3 do CLAUDE.md

O cliente pede em dinheiro, o entregador não tem troco, e o pagamento sai no cartão da maquininha. Com um campo só, a informação de que era preciso levar troco desaparece — e com ela some a medição de acerto do preparo de caixa, que é justamente o que este produto promete resolver. Nenhuma validação pode exigir que os dois coincidam.

O registro do que aconteceu é a `Liquidacao`, lista somente-inserção dentro do agregado do pedido:

Liquidacao	
└ metodoLiquidado	DINHEIRO CARTAO PIX NAO_LIQUIDADO
└ valorEfetivo	Money – zero quando NAO_LIQUIDADO
└ situacao	CONFIRMADA AGUARDANDO_CONFIRMACAO
└ momento	timestamp
└ responsavelCustodia	ENTREGADOR ESTABELECIMENTO PLATAFORMA
└ registradoPor	usuário
└ referenciaExterna	txid do Pix, NSU do cartão null
└ motivo	obrigatório quando NAO_LIQUIDADO

Três regras que não são óbvias:

1. **Pix só é confirmado por webhook do provedor.** Comprovante em imagem apresentado ao entregador não muda situação nenhuma — e isso **não impede** a conclusão. O pedido é entregue; a liquidação é que fica pendente, e aparece no fechamento como tal.
2. **Pagamento parcial gera duas liquidações**, não uma com valor menor. Metade em dinheiro e metade no cartão é o caso real.
3. **NAO_LIQUIDADO não bloqueia a conclusão.** O pedido foi entregue e não foi pago — os dois fatos são verdadeiros e ambos precisam ficar registrados. A cobrança vira assunto do estabelecimento com o cliente, não um estado preso.

7.3 Troco

trocoPara	informado pelo cliente	– "tenho uma nota de 50"
trocoDevido	= trocoPara – total	– derivado, nunca gravado por cima

Declarar troco para um valor menor que o total **impede o fechamento do pedido**: não se aceita um pagamento que já se sabe insuficiente, volta-se ao cliente com a pergunta. Campo nulo com método dinheiro significa "valor exato" — que é diferente de "não informou".

E a sobra por falta de moeda — o entregador não tem os trinta centavos — é registrada como **ajuste de arredondamento**, com esse nome. Nunca como gorjeta. Gorjeta é decisão do cliente e tem campo próprio; tratar as duas como a mesma coisa corrompe tanto o extrato do entregador quanto a medição de troco.

7.4 Congelamento

No instante em que o pedido é recebido, ele **copia** — não referencia:

COPIADO	DE	SE NÃO FOSSE CONGELADO
nome, precoBase do item	catálogo	reajuste de preço reescreveria pedidos antigos
opções escolhidas: nome e acréscimo	catálogo	"sem cebola" sumiria se a opção fosse removida
stockControlledSnapshot	catálogo	marco 10 exigiria migração de dados
nomeAreaSnapshot, taxaSnapshot	merchant	reajuste da taxa do bairro mudaria o histórico
enderecoTextual	cliente	cliente edita o endereço, o pedido antigo muda de destino
discount (parcela de retirada)	merchant	reajuste do desconto da loja mudaria pedidos passados

Congelar o **nome** do grupo e da opção, não só o preço, é o que permite reimprimir um pedido de seis meses atrás com o texto que o cliente viu, mesmo que o comerciante tenha renomeado tudo desde então. Depois do congelamento o pedido não consulta mais catálogo nem cadastro para nada que afete valor.

7.5 Ajustes: correção é lançamento, não edição

```
totalEfetivo = total + Σ ajustes.delta    – derivado, nunca gravado
```

O `total` permanece exatamente como foi fechado, para sempre. Substituição de item, remoção, arredondamento por falta de moeda e correção posterior são todos `Ajuste` — com tipo, delta, motivo obrigatório, autor e momento. É o que permite responder "quanto este pedido custava quando o cliente confirmou" seis meses depois, e é o que impede que uma correção silenciosa apague uma divergência de caixa que precisava aparecer.

A ARMADILHA QUE VAI SER COMETIDA PELO MENOS UMA VEZ

Nenhum relatório soma total para dizer quanto entrou. O que entrou é Σ `Liquidacao.valorEfetivo`; o que o pedido vale hoje é `totalEfetivo`. Isto precisa de um teste que pegue, porque a leitura errada é a intuitiva.

7.6 O preço não varia por modalidade

Muita loja cobra mais na entrega que no balcão. Aqui não: existe um **desconto de retirada** em valor fixo, configurado no estabelecimento e aplicado no `discount` no fechamento. O catálogo tem um preço só, e a operação de cotação não recebe modalidade.

O motivo é a ordem da conversa. Se o preço dependesse da modalidade, ela teria de ser conhecida **antes do primeiro número dito** — e no WhatsApp isso transforma "entrega ou retirada?" de pergunta de fechamento em pergunta de abertura. O custo se mede em turnos, e turno tem preço.

COMPONENTE	ENTREGA	RETIRADA
itemsSubtotal	48,00	48,00
deliveryFee	7,00	0,00 – I10
discount	0,00	5,00 – I12
total	55,00	43,00

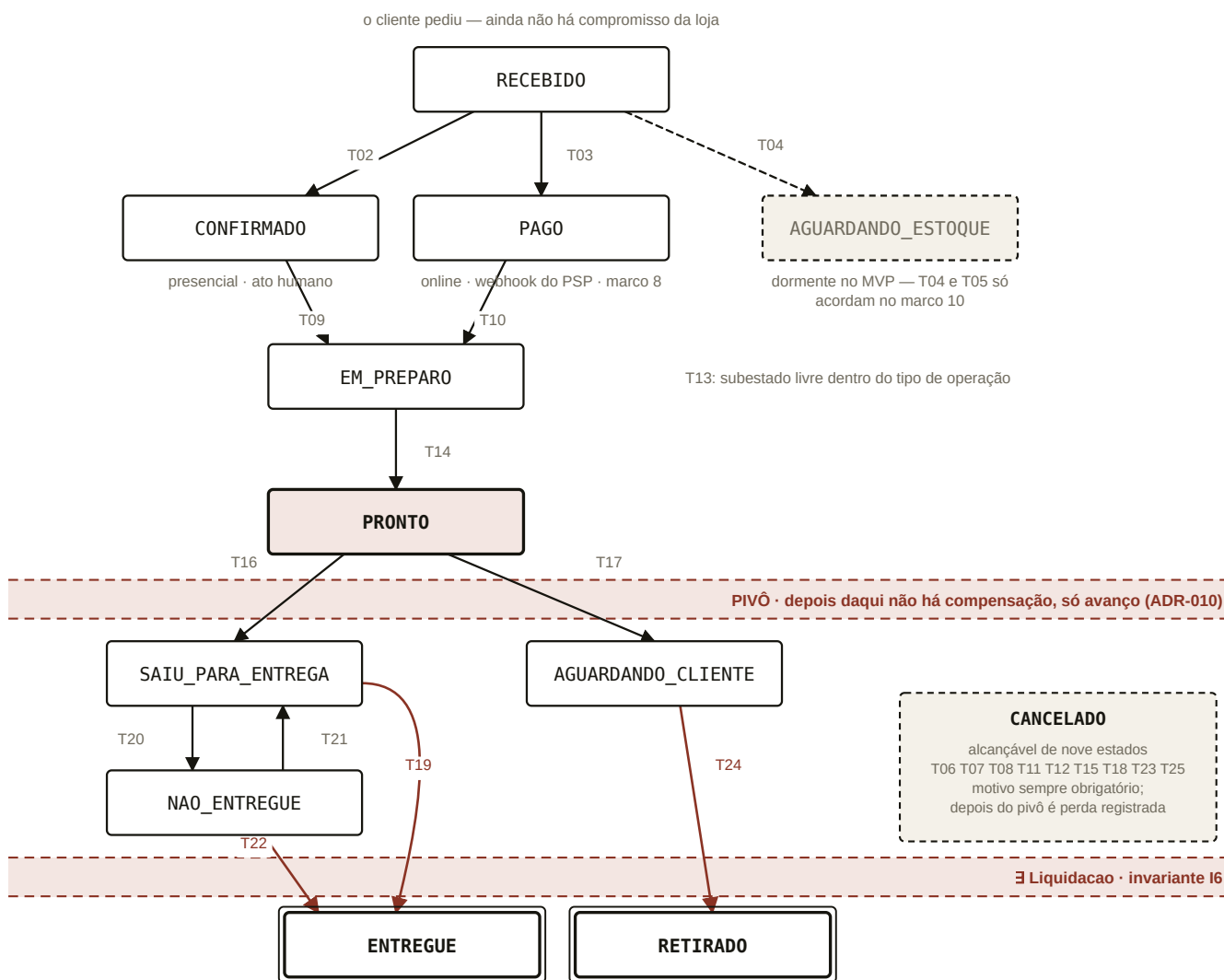
Doze reais de diferença, e o cliente consegue apontar de onde vem cada real. Num modelo de preço por modalidade ele veria os dois totais sem decomposição — e surpresa no valor é a pior coisa que um sistema de pedido pode fazer.

Vale registrar por que o argumento econômico do mercado não transfere. Cardápio com dois preços é universal nos marketplaces porque eles tiram entre 12% e 27% **do item**; o comerciante embute a comissão no preço de entrega. Aqui não há comissão sobre o item: o custo da entrega é a remuneração do entregador, e quem paga a entrega é o cliente, na taxa por área. A força que torna o cardápio duplo obrigatório lá não atua aqui.

SEÇÃO 8

O ciclo do pedido

Doze estados e vinte e cinco transições. A tabela é a fonte de verdade, e não o diagrama: diagrama não se percorre num teste parametrizado.



Vinte e cinco transições. Todo par (de, para) ausente da tabela lança exceção de domínio — HTTP 409. Borda dupla: estado terminal.

FIGURA 3 — A máquina de estados do pedido. As duas faixas horizontais são as regras que governam tudo o que as cruza: o pivô da saga e a exigência de liquidação registrada.

8.1 O portão de compromisso

RECEBIDO não obriga ninguém. É o cliente dizendo o que quer. O compromisso do **estabelecimento** é um estado à parte, com duas entradas que convergem para o mesmo fluxo de preparo: **CONFIRMADO** no presencial, por ato humano, e **PAGO** no online, por webhook do provedor.

Não existe atalho de **RECEBIDO** para **EM_PREPARO**. O compromisso é registrado mesmo quando o dono aceita e começa no mesmo segundo, porque é ele que separa "cliente pediu" de "a loja

assumiu".

O QUE CONFIRMADO NÃO É

É ato humano, não fato verificável. A plataforma não retém valor, não estorna e não bloqueia (P1). Todo controle contra abuso — limite por histórico, teto para cliente sem cadastro, bloqueio por reincidência — é **comercial**, e precisa ser registrado como tal. Nenhum deles é garantia técnica, e chamá-los assim cria uma falsa sensação de proteção onde não há.

8.2 A tabela de transições

Toda combinação (*de, para*) ausente desta tabela é inválida e lança exceção de domínio, que vira HTTP 409.

#	DE	PARA	GATILHO	GUARDA E EFEITO
T01	—	RECEBIDO	cliente confirma por botão	itens vendáveis · área atendida · invariantes de valor · regra do troco. Congela itens, taxa, área e desconto
T02	RECEBIDO	CONFIRMADO	estabelecimento aceita	loja aberta ou aceite manual explícito · ALTERAR_STATUS
T03	RECEBIDO	PAGO	webhook do PSP	assinatura válida · txid correlacionado. Marco 8
T04	RECEBIDO	AGUARDANDO_ESTOQUE	aceite com item controlado	falso no MVP — dormente até o marco 10
T05	AGUARDANDO_ESTOQUE	CONFIRMADO	reserva confirmada	dormente
T06	AGUARDANDO_ESTOQUE	CANCELADO	reserva rejeitada	ITEM_INDISPONIVEL
T07	RECEBIDO	CANCELADO	estabelecimento recusa	motivo obrigatório
T08	RECEBIDO	CANCELADO	cliente desiste	CLIENTE_DESISTIU
T09	CONFIRMADO	EM_PREPARO	produção iniciada	define subestado inicial
T10	PAGO	EM_PREPARO	produção iniciada	define subestado inicial
T11	CONFIRMADO	CANCELADO	cancelado antes do preparo	motivo e autor · sem perda de insumo
T12	PAGO	CANCELADO	cancelado antes do preparo	motivo e autor · dispara devolução
T13	EM_PREPARO	EM_PREPARO	operador muda subestado	subestado ∈ conjunto do tipo de operação · nenhum evento
T14	EM_PREPARO	PRONTO	produção concluída	pivô da saga
T15	EM_PREPARO	CANCELADO	cancelado durante o preparo	motivo e autor · perda parcial registrada
T16	PRONTO	SAIU_PARA_ENTREGA	entregador retirou na loja	modalidade ENTREGA · entrega atribuída · entregador com jornada aberta
T17	PRONTO	AGUARDANDO_CLIENTE	disponibilizado no balcão	modalidade RETIRADA

#	DE	PARA	GATILHO	GUARDA E EFEITO
T18	PRONTO	CANCELADO	cancelado após o pivô	motivo e autor · perda total, não compensação
T19	SAIU_PARA_ENTREGA	ENTREGUE	entrega confirmada	≡ Liquidacao (I6) · evento carrega a liquidação
T20	SAIU_PARA_ENTREGA	NAO_ENTREGUE	tentativa frustrada	motivo obrigatório
T21	NAO_ENTREGUE	SAIU_PARA_ENTREGA	nova tentativa	—
T22	NAO_ENTREGUE	ENTREGUE	resolvido na hora	≡ Liquidacao
T23	NAO_ENTREGUE	CANCELADO	desistência após tentativas	motivo e autor · item retorna à loja
T24	AGUARDANDO_CLIENTE	RETIRADO	cliente retirou	≡ Liquidacao (I6)
T25	AGUARDANDO_CLIENTE	CANCELADO	cliente não apareceu	motivo obrigatório · perda total

O que não existe, e a ausência é deliberada

- nenhuma transição **saindo** de ENTREGUE, RETIRADO ou CANCELADO;
- nenhuma de SAIU_PARA_ENTREGA de volta para PRONTO;
- nenhum atalho de CONFIRMADO para PRONTO — sem preparo registrado não há tempo de preparo mensurável;
- nenhuma transição para ENTREGUE ou RETIRADO sem liquidação.

É a regra mais importante da tabela. É o que separa este sistema do caderno.

docs/dominio/pedido.md §3

Como isto vira teste

A tabela é dado, não código. O teste é parametrizado sobre ela, e cobre seis coisas: toda transição é aceita quando a guarda é satisfeita; toda transição é rejeitada quando a guarda é violada; **todo par fora da tabela lança exceção**; nenhum estado não-terminal fica sem saída; todo estado é alcançável a partir de T01; e nenhum pedido entra nos estados dormentes enquanto o MVP durar.

O terceiro é o que costuma faltar. Testar só o caminho feliz deixa a máquina de estados aberta, e descobre-se em produção que cancelado voltava a sair para entrega.

8.3 As doze invariantes do agregado

Verificadas na construção e em toda transição — não em teste de integração.

#	INVARIANTE	ONDE QUEBRA SE NÃO EXISTIR
I1	total == itemsSubtotal + deliveryFee + tip - discount	cobrança diverge do extrato
I2	itemsSubtotal == $\sum$ item.subtotal	relatório de vendas não bate

#	INVARIANTE	ONDE QUEBRA SE NÃO EXISTIR
I3	$\text{todo componente} \geq 0 \cdot \text{discount} \leq \text{itemsSubtotal} + \text{deliveryFee}$	total negativo
I4	método dinheiro com troco informado $\rightarrow \text{trocoPara} \geq \text{total}$	entregador na porta sem troco possível
I5	total imutável após RECEBIDO; correção só por Ajuste	perde-se o valor que o cliente confirmou
I6	ENTREGUE ou RETIRADO $\rightarrow \exists \text{ Liquidacao}$	o produto inteiro perde a razão de existir
I7	todos os itens do mesmo estabelecimento	pedido multiloja, fora de escopo
I8	EM_PREPARO $\leftrightarrow$ subestado não nulo	subestado órfão na tela
I9	CANCELADO $\rightarrow$ motivo não nulo	"cancelados" vira número sem diagnóstico
I10	retirada $\rightarrow$ deliveryFee zero e área nula	cobra-se entrega de quem foi buscar
I11	nenhum estado não-terminal sem transição de saída	pedido preso para sempre
I12	retirada $\rightarrow$ discount é o desconto congelado; entrega $\rightarrow$ discount zero	desconto prometido some no fechamento

I12 vale enquanto `discount` tiver origem única. No dia em que existir cupom, ela é a primeira a ser revisitada.

I7 é imposta em dois lugares, não só no agregado. O carrinho já recusa item de outro estabelecimento ao ser acrescentado — mensagem clara, antes de o cliente montar um pedido inteiro. A invariante é a segunda guarda, porque o carrinho é estado externo e não se confia em validação que já passou.

SEÇÃO 9

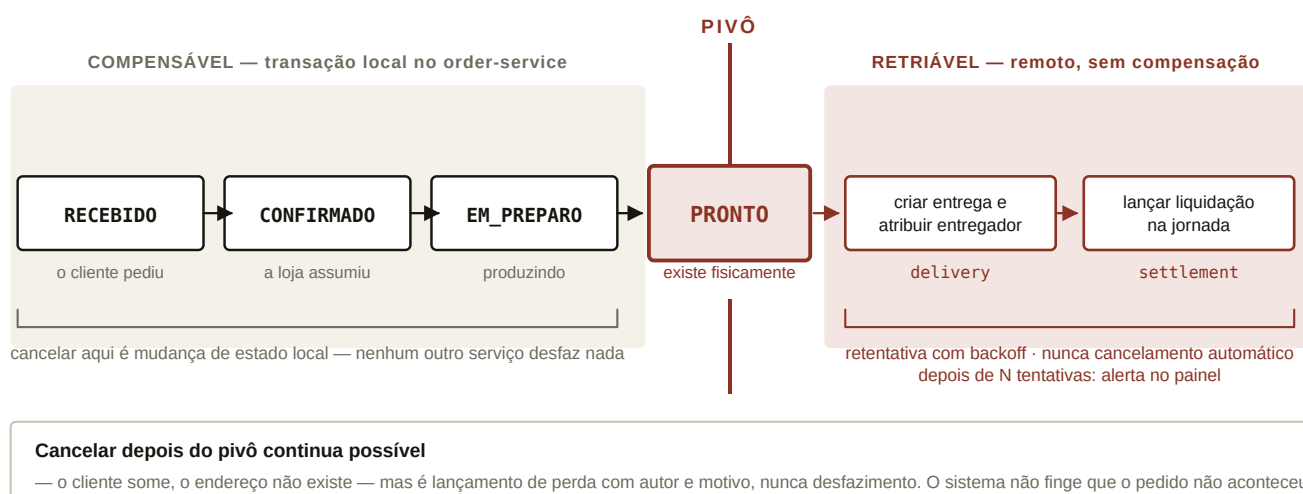
A saga e o pivô

Uma saga precisa de três coisas para ser desenhável: saber quais passos são **compensáveis**, qual é a **transação-pivô** — a partir dela não se desfaz mais, só se avança — e quais são **retriáveis**.

Na versão 1.0 o pivô era a cobrança: o cartão passou, agora vai. Aqui não há cobrança para passar. O cliente paga em dinheiro na porta, ou no cartão da maquininha, ou por Pix confirmado depois — sempre **depois** de a comida estar pronta e ter saído.

O ponto de não-retorno deste sistema não é financeiro. É produtivo.

ADR-010



O pagamento não é perna da saga. Na maioria dos pedidos o dinheiro nunca passa pela plataforma: o cliente paga na porta, depois de a comida existir (P1).

FIGURA 4 — A saga do pedido. À esquerda do pivô, cancelar é mudar um estado numa transação local. À direita, não existe compensação barata: existe avançar, com retentativa, até um estado terminal.

9.1 Por que o pivô é **PRONTO**, e o critério é estrutural

Antes de **PRONTO**, cancelar um pedido é uma transação local: muda-se um estado, registra-se a causa, acabou. Nenhum outro serviço precisa desfazer nada, porque nenhum outro serviço foi acionado. Em **PRONTO**, o pedido entra no fluxo distribuído — cria-se a entrega, atribui-se o entregador, começa a custódia de valores.

Foi considerado seriamente colocar o pivô no início do preparo, onde o custo é de fato incorrido. Rejeitado porque o critério escolhido é **estrutural, não econômico**: o pivô marca onde a transação deixa de ser local. Um critério econômico exigiria decidir qual percentual de custo já incorrido "conta", que não é pergunta de arquitetura.

9.2 O pagamento sai da saga

O `payment` deixa de ser passo orquestrado e vira fronteira com o provedor. Não há estado de pagamento pendente no caminho comum, não há janela de dez minutos, não há timeout de pagamento cancelando pedido.

A única exceção é o Pix com confirmação por webhook, e mesmo ela não é perna de saga: é um evento assíncrono que muda o estado de uma liquidação, não o estado do pedido. **Um pedido entregue com Pix ainda não confirmado é um pedido entregue.** Prender a entrega ao webhook seria travar o entregador na porta do cliente por causa de latência de terceiro.

9.3 As duas pernas remotas

PERNA	SERVIÇO	SE FALHAR
Criar entrega e atribuir entregador	delivery	retentativa com backoff; após N tentativas, alerta no painel — nunca cancelamento automático
Lançar liquidação na jornada	settlement	retentativa; a liquidação já está gravada no pedido, a jornada é que precisa alcançá-la

Nenhuma das duas é compensável, e isso é intencional: depois do pivô, compensar significaria "des-entregar", que não existe.

9.4 Os custos assumidos

NÃO HÁ GARANTIA DE RECEBIMENTO

O pedido é produzido antes de qualquer pagamento, e o cliente pode não pagar. Isso não é defeito do desenho — é a realidade do negócio, e o produto responde a ela **medindo**: `NAO_LIQUIDADO` é registro de primeira classe e aparece no fechamento. Nenhuma arquitetura resolve calote; esta se recusa a escondê-lo.

Duas pernas retriáveis sem compensação exigem **fila morta e procedimento operacional**. "Tentar para sempre" sem um lugar onde a mensagem morre e alguém olha é como se perde pedido em silêncio — e isso ainda não existe (seção 18).

Os motivos de cancelamento são enumerados, não texto livre. Sem isso, "pedidos cancelados" no relatório é um número sem diagnóstico. Com isso, cliente ausente alto vira conversa sobre confirmação no canal, e falta de entregador alto vira decisão de contratação.

Autorização contextual

Permissão não pertence ao usuário.

Pertence ao vínculo entre o usuário e o estabelecimento.

docs/dominio/estabelecimento.md §2 · ADR-011

Jorge é administrador da pizzaria da Marli e, no mercadinho do Sérgio, é entregador sem acesso a nada. Não são dois usuários — é o mesmo usuário com dois vínculos. A pergunta que o sistema responde nunca é "o que este usuário pode fazer"; é "**o que este usuário pode fazer nesta loja**".

10.1 O que o token carrega, e o que não carrega

CARREGA	NÃO CARREGA
identidade do usuário	permissões
validade, emissor, assinatura	papel
—	lista de estabelecimentos

Permissão dentro do token quebra de duas formas. Fica **velha** — a Marli revoga o acesso do Rafa às 20h e o token dele continua valendo até expirar. E é **por estabelecimento** — um token que carregasse todas as permissões de todas as lojas cresceria com o número de vínculos e vazaria a lista de lojas do usuário em cada requisição.

10.2 O cache é local ao consumidor e vive em processo

Cada serviço mantém a sua própria cache em memória, atrás de uma porta. Não há Redis nesse caminho — a cache existe para evitar uma ida à rede, e colocá-la no Redis trocaria uma ida à rede por outra. Uma chave compartilhada também foi rejeitada: transformaria o Redis num canal por onde um serviço lê dado de outro sem passar pela porta dele.

PARÂMETRO	VALOR	MOTIVO
Chave	usuário × loja	a pergunta é sempre esse par
TTL de resposta positiva	60 s	ver abaixo
TTL de resposta negativa	10 s	também se cacheia "não tem vínculo", senão uma varredura bate direto no serviço. Curto porque acesso recém-concedido não pode demorar
Tamanho máximo	10 000	teto de memória previsível, descarte por menos-recentemente-usado
Timeout da chamada	300 ms	além disso, a requisição do usuário já está lenta
Disjuntor	por falhas consecutivas	evita fila de requisições esperando um serviço morto

O TTL de sessenta segundos é três coisas ao mesmo tempo

- **Segurança:** é o pior caso de acesso indevido depois de uma revogação cujo evento se perdeu. Para um sistema onde ninguém movimenta dinheiro sem registro, é aceitável.
- **Carga:** cada serviço consulta uma vez por minuto por usuário, em vez de uma vez por clique. Reduz a carga em mais de 99%.
- **Disponibilidade:** e aqui está o ponto que costuma passar batido — o TTL *também* é a tolerância a queda. Se o serviço de autorização cair por quarenta segundos, a maioria das requisições nem percebe. Não é preciso inventar um modo degradado separado: ele já existe, e dura exatamente o TTL.

10.3 Invalidação por evento, TTL como rede de segurança

A cada criação, alteração, suspensão ou remoção de membro, o serviço publica um evento e cada consumidor remove a entrada correspondente da sua cache. Efeito em segundos; o TTL cobre o caso do evento perdido — e evento **se perde**, por isso as duas coisas existem.

A PRIMEIRA COISA A QUEBRAR QUANDO HOUVER DUAS INSTÂNCIAS

Cache em processo com N instâncias significa N caches. O evento vai para um *exchange* fanout, e cada instância precisa de uma **fila exclusiva e temporária**. Com fila compartilhada, só uma instância invalida e as outras seguem com permissão revogada até o TTL — silenciosamente. No MVP há uma instância por serviço e isso é invisível.

10.4 Indisponível é negado

Cache expirada e serviço de autorização fora: **403**. Não se serve entrada vencida. Uma janela de graça foi considerada e rejeitada — ela inverte o modo de falha: em vez de "ninguém entra", vira "todo mundo continua entrando com as permissões antigas", e a revogação para de funcionar exatamente quando o sistema está instável.

Fail-closed que abre sob pressão não é fail-closed.

ADR-011

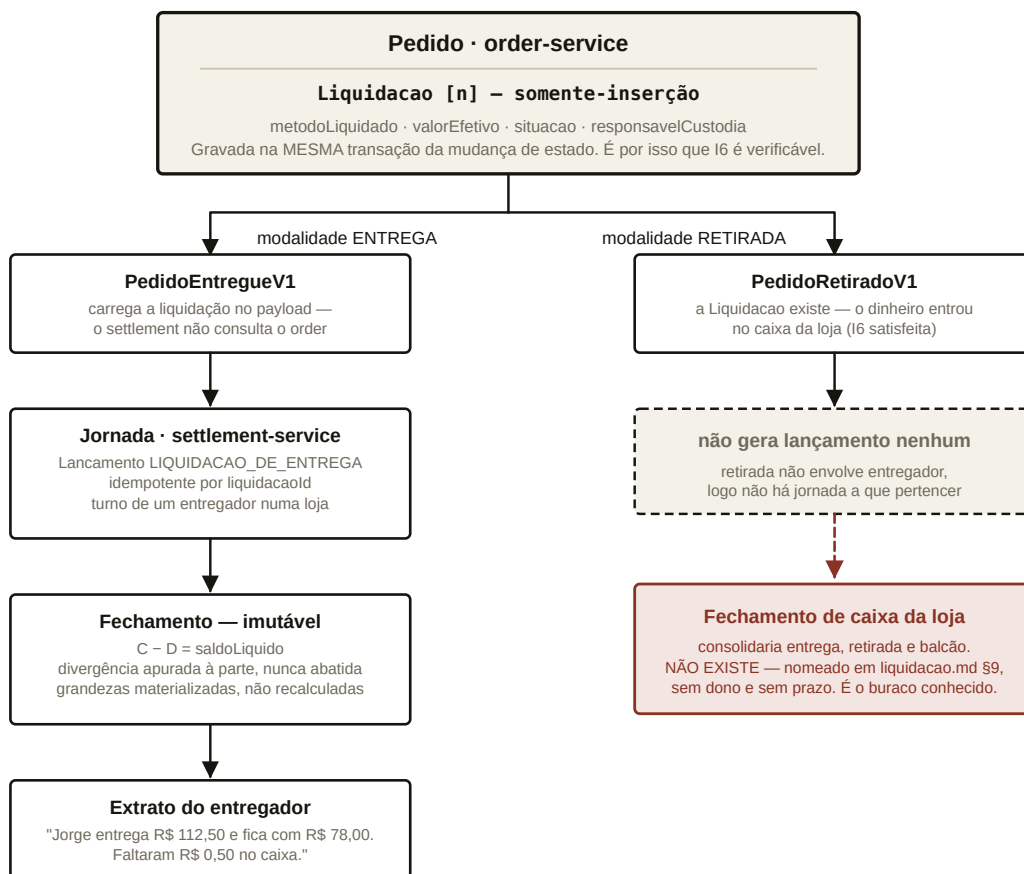
O custo é real e assumido: se o serviço de autorização cair por mais de um minuto, a plataforma inteira para de autorizar. A resposta a isso é disponibilidade — réplicas, verificação de saúde, alerta — não relaxar a regra.

10.5 Três regras que se esquecem

- **O 403 é indistinguível de "não existe"**. Sem vínculo, com vínculo mas sem a permissão, ou loja inexistente: a mesma resposta. Código diferente por caso transforma o erro num scanner de estabelecimentos.
- **Ninguém concede permissão que não possui**, e a verificação é feita *duas* vezes: no convite e no aceite. Entre um e outro podem passar dias, e o convidante pode ter perdido a permissão que estava concedendo. A revogação segue a mesma simetria — sem ela, um gerente rebaixa colegas usando uma permissão que ele próprio não tem.
- **Um estabelecimento sempre mantém pelo menos um administrador ativo**. A operação que violaria isso falha com mensagem explícita, nunca em silêncio.

O fechamento de expediente

Às onze da noite, o dono precisa saber quanto dinheiro deveria estar na mão do Jorge, quanto está, e quem deve para quem. Hoje ele descobre no papel e a conversa termina em desconfiança. O sistema tem que chegar ao mesmo número por um caminho que os dois consigam conferir.



Dois objetos, nomes parecidos

A Liquidacao vive no order-service, dentro do Pedido, e registra que o dinheiro foi recebido — I6 a exige, sempre.
O Lancamento de tipo LIQUIDACAO_DE_ENTREGA vive no settlement-service, dentro da Jornada, e é o reflexo dela no acerto do entregador — J1 o restringe.
Toda entrega produz os dois. Toda retirada produz só o primeiro.

Nenhum relatório soma total. O que entrou é $\sum Liquidacao.valorEfetivo$; o que o pedido vale hoje é $totalEfetivo = total + \sum ajustes.delta$.

FIGURA 5 — O caminho do dinheiro, do registro no pedido ao extrato do entregador. A coluna da direita mostra o que acontece na retirada — e onde o sistema tem um buraco conhecido.

11.1 A jornada

Uma jornada é o turno de **um entregador em um estabelecimento**. O mesmo entregador em duas lojas no mesmo dia tem duas jornadas independentes, com remunerações possivelmente diferentes.

A remuneração é **congelada na abertura**. Se o comerciante alterar a diária do Jorge às 22h, a jornada aberta às 18h continua valendo o que valia quando começou. Sem isso, o extrato muda depois de o entregador já ter visto o número — e é precisamente aí que a confiança se perde.

Tudo que acontece na jornada é **Lancamento**, **somente-inserção**: nada é editado, nada é apagado. Entrega concluída, liquidação de entrega, adiantamento, não liquidado. A chave natural de idempotência é o identificador da liquidação — mensagem repetida não gera segundo lançamento, porque contar a mesma entrega duas vezes infla a comissão e destrói o fechamento.

11.2 As fórmulas

F	fundoDeTrocoEntregue	(da abertura)
E	contagem de lançamentos ENTREGA_CONCLUIDA	
LD	Σ valorEfetivo · dinheiro · confirmada	
LC	Σ valorEfetivo · cartão · confirmada	
LP	Σ valorEfetivo · Pix · confirmada	
LA	Σ valorEfetivo · aguardando confirmação	(Pix pendente)
LN	Σ valorNaoRecebido	
GD	Σ gorjeta · dinheiro · confirmada	
GC	Σ gorjeta · cartão ou Pix · confirmada	
A	Σ valor · adiantamento	

Conferência de caixa

```
dinheiroDaLoja    = LD - GD
dinheiroEsperado  = F + dinheiroDaLoja
divergencia       = dinheiroConferido - dinheiroEsperado
```

O entregador entrega **o dinheiro da loja**. A gorjeta em dinheiro é dele e não entra na conferência — contá-la obrigaria a revistar o bolso de alguém todo dia, o que é péssimo produto e pior relação de trabalho.

POR QUE O FUNDO DE TROCO ENTRA E O TROCO DEVOLVIDO NÃO

O entregador recebe R\$ 50 num pedido de R\$ 38 e devolve R\$ 12 do próprio bolo. O saldo do bolso move $+50 - 12 = +38$, que é exatamente o valor efetivo. O troco se anula sozinho. O fundo só existe para o caso em que ele não consegue fazer o troco com o que já tem em mãos.

Extrato

```
Créditos    C = valorDiaria + (E × comissaoPorEntrega)
              + (E × taxaFixaPorEntrega) + GC

Débitos     D = dinheiroDaLoja + F + A

saldoLiquido = C - D
```

Saldo positivo: **a loja paga** esse valor ao entregador. Saldo negativo: **o entregador entrega** esse valor à loja, e fica com o restante do dinheiro como pagamento — que é como a operação já funciona no balcão.

Três grandezas não entram em nenhuma das colunas: o Pix confirmado (caiu direto na conta da loja, o entregador nunca teve custódia), o Pix pendente (ainda não é receita nem calote — tem coluna própria no extrato) e o não liquidado (a loja não recebeu, e não é dívida do entregador).

11.3 Exemplo trabalhado

Jorge, turno da noite. Diária de R\$ 80,00 mais comissão de R\$ 4,00 por entrega. Fundo de troco de R\$ 50,00. Seis entregas.

LD = 145,50	GD = 5,00	LC = 115,00	GC = 4,00	LA = 38,00	A = 30,00	E = 6
dinheiroDaLoja	= 145,50 - 5,00		= 140,50			
dinheiroEsperado	= 50,00 + 140,50		= 190,50			
dinheiroConferido (contado no balcão)			= 190,00			
divergencia	= 190,00 - 190,50		= -0,50	← falta R\$ 0,50		
C = 80,00 + (6 × 4,00) + 0,00 + 4,00			= 108,00			
D = 140,50 + 50,00 + 30,00			= 220,50			
saldoLiquido = 108,00 - 220,50			= -112,50			

Leitura para o comerciante: "Jorge entrega R\$ 112,50 e fica com R\$ 78,00 de pagamento. Faltaram R\$ 0,50 no caixa. R\$ 38,00 de Pix ainda aguardam confirmação do banco."

Confere: ele tem R\$ 190,50 de dinheiro da loja, entrega R\$ 112,50, retém R\$ 78,00 — que é exatamente créditos menos adiantamentos, a remuneração ainda não antecipada.

11.4 Divergência nunca é compensada em silêncio

A divergência **não** entra no saldo líquido. São números diferentes, com naturezas diferentes: um é acerto contratual, o outro é um erro que precisa aparecer.

O comerciante decide o que fazer com ela — absorver ou descontar — e a decisão é um ajuste explícito, com autor e motivo. Se o sistema abatesse a falta do pagamento automaticamente, a divergência sumiria do relatório e a métrica de acerto de caixa passaria a mostrar zero para sempre.

Sobra silenciosamente absorvida é o mesmo defeito que falta silenciosamente absorvida: some o sinal de que algo está errado no preparo de caixa.

11.5 O fechamento é um documento, não uma consulta

Ao confirmar, grava-se um fechamento **imutável** com todas as grandezas materializadas — não recalculadas na leitura. Se o extrato fosse recalculado a cada leitura, uma mudança futura na fórmula reescreveria retroativamente extratos que entregador e comerciante já assinaram.

Quando um webhook de Pix atrasa e chega com a jornada já fechada, três regras valem: o fechamento **não é alterado, nunca**; cria-se um ajuste de liquidação tardia com o valor; e o registro do Pix pendente permanece no fechamento original, porque é o histórico de que aquele valor esteve pendente naquele momento.

11.6 Quem absorve o não liquidado

O cliente não pagou. O entregador fez a entrega. **Padrão: o entregador recebe a comissão e não é debitado do valor.** O trabalho foi feito; o risco de crédito é do estabelecimento, que decidiu vender sem receber antes.

Existe uma política configurável por estabelecimento que inverte isso. O que a arquitetura decide não é a política — é que ela seja explícita, configurável e visível no extrato, em vez de virar uma subtração que o entregador descobre no fim do mês.

Catálogo, entrega e conversa

Os outros três domínios, no que cada um tem de menos óbvio.

12.1 Catálogo — o dono do preço

Nenhum outro serviço calcula quanto custa alguma coisa; todos perguntam. É por isso que a regra "o valor cobrado vem do pedido" é cumprível: o valor vem do pedido, e o pedido vem daqui.

```
precoUnitario = precoBase +  $\Sigma$  acrescimo das opções escolhidas  
subtotal      = precoUnitario  $\times$  quantidade
```

Acréscimo negativo é legítimo: o comerciante que desconta R\$ 2 por tirar o queijo está exercendo política de preço, não erro de digitação. O que a invariante protege é o resultado — o preço unitário tem de ser maior que zero.

Vendabilidade derivada

```
vendavel = estadoDePublicacao == ATIVO  
           $\wedge$  disponibilidade  $\in$  { DISPONIVEL, ACABANDO }  
           $\wedge$   $\forall$  grupo com minEscolhas  $\geq 1$  :  
              contar(opções disponíveis no grupo)  $\geq$  minEscolhas
```

Pizza disponível cujo grupo obrigatório "Tamanho" está com **todos** os tamanhos esgotados **não é vendável** — não há seleção válida possível. Sem esta regra o produto aparece no cardápio, o cliente escolhe, e a cotação recusa no fim do fluxo. No WhatsApp isso custa turnos, e turno custa dinheiro.

E é **derivado, nunca gravado**: gravado, desatualiza no instante em que uma opção muda de estado.

A reativação automática não é um cron

Produto marcado como esgotado hoje volta sozinho na abertura do próximo expediente. A tentação é um job à meia-noite, e **está errado**: a pizzeria abre às 18h e fecha às 2h; à meia-noite ela está vendendo, e um job zeraria a calabresa que acabou às 23h — voltando a oferecer o que não existe, no meio do pico.

O gatilho é o **evento de abertura do estabelecimento**, com três consequências que precisam estar no código: retomada de pausa não reativa nada (pausar às 20h e retomar às 20h30 é o mesmo expediente); o identificador do expediente é obrigatório, porque é ele que torna a operação idempotente quando a mensagem se repetir; e loja que não abriu não reativa — segunda-feira fechada não zera o que acabou no domingo, que é o comportamento certo, porque o estoque físico também não se repõe sozinho.

Cotação

A cotação é o **ponto de verdade**, não a listagem. O cardápio exibido pode estar em cache de segundos atrás; a cotação lê o estado corrente. Entre ver a pizza na tela e confirmar o pedido, a calabresa pode ter acabado — e é a cotação que descobre isso.

A distinção entre 400 e 409 não é preciosismo: 400 é erro do chamador, pedido malformado, e vira alerta; 409 é o estado do mundo que mudou, e volta ao cliente com a lista do que caiu, para reconfirmação. E a cotação **não reserva nada**: responde quanto custa e se está disponível agora.

12.2 Entrega — o serviço que fica sabendo

O **delivery-service** não declara uma entrega concluída. Ele fica sabendo.

docs/dominio/entrega.md §1

A razão é a invariante I6: a liquidação vive dentro do agregado do pedido. Se este serviço pudesse marcar uma entrega como concluída por conta própria, existiria um caminho para concluir sem passar pelo portão de liquidação — e o portão deixaria de ser garantia. Na prática o entregador toca um botão só; esse toque vira a transição no pedido, com a liquidação junto.

Duas máquinas que o PRD lista como uma

"Retornando" e "disponível na loja" não descrevem uma entrega — descrevem o **entregador**. Uma entrega concluída está concluída; quem volta para a loja e fica disponível é a pessoa. Isso não é purismo: o rodízio ordena por ordem de retorno à loja, e ordem de retorno é atributo do entregador. Enquanto as duas máquinas estiverem fundidas, o rodízio não tem por onde ordenar.

Não existe aceite

A atribuição é uma **ordem de serviço, não uma oferta**. O entregador é funcionário da casa, e o comerciante decide quem leva o quê. Isso elimina de uma vez oferta, janela de aceite, timeout, reoferta, fila de candidatos, penalidade por recusa e todo o desenho de leilão que a v1.0 tinha. Se o entregador não pode levar, ele fala com o operador, que reatribui — a conversa é humana; o sistema registra o resultado.

O rodízio **sugere**; o operador confirma com um toque e pode ignorar. Quando ignora, isso é gravado — não para vigiar ninguém: sem esse dado, "o rodízio não está funcionando" é uma discussão sem evidência, e a resposta quase sempre é que ele está funcionando e sendo contornado por um bom motivo que ninguém registrou.

Concluir a última entrega **não** torna o entregador disponível: ele fica retornando, e só o registro do retorno o devolve à loja. Inferir disponibilidade a partir da conclusão quebraria o rodízio de um jeito específico e injusto — quem entrega perto voltaria "disponível" antes de

estar na loja e receberia mais pedidos que quem entrega longe. O critério passaria a ser distância, que é justamente o que a premissa P5 tirou do sistema.

12.3 Conversa — a fronteira de confiança

Tudo que a interpretação devolve é **entrada não confiável**, do mesmo modo que um corpo de requisição HTTP. Antes de qualquer uso, todo identificador proposto é validado: o produto existe, pertence a este estabelecimento e está ativo; a opção pertence ao produto; a área pertence a esta loja e está ativa; a quantidade é inteira e está dentro do teto.

Identificador que não passa é descartado em silêncio para o modelo e vira um pedido de esclarecimento para o cliente — **nunca** um erro técnico na tela dele.

Confirmação: a regra que não tem exceção

O compromisso do cliente é um toque em botão interativo com identificador conhecido. Nada mais. Não valem reação com emoji, texto livre interpretado como "sim" — "pode ser", "acho que sim", "tá bom" não são compromisso, e um modelo que os classifica como tal cria pedidos que o cliente não fez —, áudio, nem silêncio depois de um resumo.

O escalonamento sobre alergia é diferente dos outros

Três dos quatro gatilhos de escalonamento toleram uma falha de detecção: o cliente insiste, o contador sobe, o teto é aritmética. **Alergia não tolera**. Uma resposta automática errada sobre a presença de amendoim num produto é um risco à saúde de alguém.

Por isso esse gatilho **não pode depender de o modelo reconhecer o assunto**. Existe um piso determinístico — uma lista de termos verificada *antes* da interpretação e independente dela. O modelo também escalona, por instrução explícita, mas ele **soma; nunca substitui** o piso.

O ERRO QUE SE PREFERE

O piso é grosseiro e vai escalonar conversas que não precisavam. Um atendente lendo uma pergunta boba custa segundos; a outra direção custa outra coisa.

Custo é requisito, não métrica

O custo tem duas naturezas, e medir só uma esconde metade: o canal cobra por **mensagem enviada**, o modelo cobra por **token**. Uma conversa com poucos turnos e um modelo caro pode custar mais que uma com muitos turnos e menu numerado. O teto configurável é sobre o custo estimado total da conversa.

Ao atingir o teto, o modo é rebaixado para determinístico e o painel é sinalizado. **Nunca se encerra a conversa** — o cliente no meio de um pedido não pode ser desligado por causa de orçamento.

Eventos e contratos

Integração entre serviços é por API síncrona, com timeout, ou por evento assíncrono. Não há terceira forma, e não há acesso ao banco alheio.

13.1 Outbox e idempotência

Quem publica precisa de outbox.

Quem consome precisa de `processed_messages`.

Invariante 7 do CLAUDE.md — sem exceção

Gravar o dado de negócio e a linha do outbox tem de ser **um** ato — ou os dois acontecem, ou nenhum. Sem isso existem dois defeitos, e ambos são silenciosos: evento publicado para um pedido que não foi gravado, ou pedido gravado cujo evento nunca sai.

É daqui que vem uma exigência de infraestrutura que parece arbitrária: o MongoDB sobe como **replica set de nó único**, porque em modo avulso ele não faz transação multi-documento — e sem transação não há outbox. O nome engana: com um nó não há redundância nenhuma. É a topologia mínima que habilita a transação, e nada além disso.

A FALHA MAIS CARA E MAIS FÁCIL DE COMETER

Sem o parâmetro `?replicaSet=rs0` na string de conexão, o driver conecta em modo avulso mesmo com o servidor configurado corretamente. E o erro não aparece no boot: aparece na primeira transação, que pode ser semanas depois, possivelmente em produção.

13.2 O envelope

Todo evento carrega identificador próprio, tipo, versão, momento de ocorrência, `correlationId` e conteúdo. O `correlationId` é código, não infraestrutura, e precisa estar no conteúdo desde o primeiro evento — acrescentá-lo depois significa revisitar todo ponto de publicação.

13.3 Regra de versionamento

Mudança incompatível cria uma nova versão do esquema; a anterior continua publicada até nenhum consumidor depender dela. Contrato de evento é mais difícil de mudar que contrato REST — o consumidor pode estar processando uma mensagem antiga que já está na fila.

O que ainda **não** está escrito é o que conta como mudança compatível, e como se sabe que nenhum consumidor depende mais da versão anterior. O sufixo de versão sem essa regra é convenção esperando ser quebrada — está na seção 18.

13.4 Os eventos que atravessam o sistema

EVENTO	PUBLICADO POR	CONSUMIDO POR, E PARA QUÊ
PedidoRecebidoV1	order	conversation — confirmar ao cliente
PedidoConfirmadoV1	order	conversation
PedidoProntoV1	order	delivery — cria a entrega, se a modalidade for entrega · conversation
PedidoSaiuParaEntregaV1	order	delivery, conversation
PedidoEntregueV1	order	settlement, delivery, conversation — carrega a liquidação no conteúdo
PedidoRetiradoV1	order	settlement, conversation
PedidoCanceladoV1	order	delivery, settlement, conversation
LiquidacaoConfirmadaV1	payment	order — muda a situação da liquidação para confirmada
VinculoAlteradoV1	merchant	todos — invalidação da cache de autorização
DisponibilidadeAlteradaV1	catalog	conversation — parar de oferecer o que acabou
abertura de expediente	merchant	catalog — reativa o esgotado de ontem · conversation
AreasDeEntregaAlteradasV1	merchant	conversation — lista de bairros
ConversaEscalonadaV1	conversation	painel — fila de atendimento humano

O **settlement** não escuta o **payment**. A apuração se alimenta dos eventos de conclusão do pedido, que carregam a liquidação, mais a confirmação tardia do Pix. Nunca de um evento do **payment** sobre o mesmo fato — dois caminhos para o mesmo dinheiro é contagem dupla.

UMA COLISÃO DE NOME AINDA ABERTA

O evento de disponibilidade do catálogo e o evento de expediente do merchant tiveram, por um tempo, o mesmo nome com significados diferentes. A renomeação foi decidida e está **parcialmente aplicada**: o conversation já consome o nome novo, e o merchant ainda publica o antigo. Está na seção 18.

Segurança

14.1 Identidade

Token JWT assinado com chave assimétrica. Só o `identity-service` emite; todos os outros validam pela chave pública publicada. As *claims* seguem o padrão do protocolo, de modo que uma migração futura para um servidor de autorização completo troque apenas o emissor — nenhum serviço validador é tocado, porque todos só conhecem a chave.

O que essa escolha custa está registrado: rotação de token de renovação, detecção de reúso e revogação passam a ser responsabilidade própria. Tokens de renovação são tratados por **família**, e o reúso de um token já consumido revoga a família inteira.

14.2 Os webhooks são públicos, e isso precisa ser explícito

O prefixo de webhooks **não exige token** — o provedor de pagamento e o provedor do canal não têm token nosso. A autenticação deles é **assinatura no corpo**, validada dentro do serviço, com processamento idempotente porque o endereço é público e a mensagem se repete.

O ERRO CLÁSSICO, NAS DUAS DIREÇÕES

Ou o filtro de autenticação bloqueia o webhook e as notificações somem em silêncio, ou a exceção é escrita larga demais e abre mais do que devia. Por isso o caminho é um prefixo próprio e explícito, e **nenhuma outra rota mora sob ele**. O filtro que libera exatamente esse prefixo ainda não foi escrito — é requisito do marco 1.

14.3 As regras que valem desde a primeira linha

- **Nenhum segredo no repositório.** Apenas um arquivo de exemplo, com nomes de variáveis. Varredura de segredos roda no CI.
- **Nenhum identificador vindo da URL é confiável** sem confronto com o usuário autenticado e as permissões no estabelecimento.
- **Valor cobrado vem do pedido.** Preço, total e taxa são sempre recalculados no servidor a partir do catálogo. Valor que chega na requisição é ignorado.
- **Timeout obrigatório em toda chamada entre serviços.** O padrão da biblioteca é esperar para sempre, e a primeira chamada sem timeout é uma indisponibilidade latente.
- **Log sem token, senha, documento, dado de pagamento ou coordenada exata.**
- **Comprovante em imagem não confirma Pix.** Só o webhook do provedor, com identificador correlacionado ao pedido. É esse pixel que decide se o golpe do comprovante falso funciona.

14.4 A fronteira com o provedor de pagamento

O `order` é dono do registro; o `payment` é dono da conversa com o provedor. Ele guarda o segredo, gera a cobrança, recebe o webhook, valida a assinatura e traduz tudo isso num evento de domínio. O que foi recebido, de quem, por qual método e sob custódia de quem é assunto do pedido.

Trocar de provedor passa a ser trocar um adaptador — o modelo de liquidação não sabe qual provedor existe. E a idempotência é feita em dois lugares por motivos diferentes: no webhook, contra o reenvio do provedor; no consumidor do evento, contra a reentrega do broker.

Privacidade e LGPD

ANTES DE QUALQUER COISA

Isto é uma decisão de engenharia, não um parecer jurídico. Os prazos abaixo são padrões com justificativa e precisam de **revisão por advogado antes do primeiro cliente real**. Onde a lei é citada, é por artigo, para que a revisão saiba onde olhar.

15.1 A base legal é execução de contrato, não consentimento

Consentimento é revogável, tem de ser granular, e sua revogação obriga a apagar. Usá-lo para o pedido significaria que um cliente poderia revogar no meio da entrega e o sistema teria de se apagar. Execução de contrato é a base correta e a mais honesta: o dado é usado para fazer o que a pessoa pediu.

CATEGORIA	BASE LEGAL
Pedido, endereço, telefone, entrega	execução de contrato
Registro fiscal e liquidação	obrigação legal
Conta de usuário do sistema	execução de contrato
Prevenção a fraude e abuso	legítimo interesse — exige teste de balanceamento documentado
Histórico do cliente no canal ("o de sempre?")	legítimo interesse — a base mais frágil; o recurso é construído isolado , para poder ser desligado se a revisão não o aceitar
Áudio guardado além da transcrição	consentimento — e por isso não guardamos

15.2 A forma mais barata de cumprir a lei é não ter o dado

NÃO GUARDAMOS	EM VEZ DISSO
coordenada geográfica exata	endereço textual e nome de bairro
áudio da mensagem	transcrição; o áudio é descartado após transcrever
número de cartão	identificador da transação, no payment
imagem de comprovante como dado financeiro	nada — imagem não confirma pagamento
telefone e nome no provedor de interpretação	só o texto necessário

O **catalog** não guardar dado pessoal é decisão, não acaso: cardápio é dado de negócio, e mantê-lo assim deixa um serviço inteiro fora do escopo de exclusão. O gateway também.

15.3 Exclusão anonimiza o operacional; o registro fiscal permanece

A colisão entre o direito do titular e a obrigação de conservar registro fiscal se resolve **separando os campos, não os registros**.

Contato, conversas e mensagens	→ EXCLUÍDOS
Endereços conhecidos	→ EXCLUÍDOS
Pedido	→ ANONIMIZADO, não excluído
└─ nome do cliente	→ "titular removido"
└─ telefone	→ nulo
└─ endereço textual	→ "endereço removido"
└─ nome da área	→ PERMANECE (bairro não identifica ninguém)
└─ todos os valores	→ PERMANECEM
└─ itens e opções	→ PERMANECEM
└─ liquidações e ajustes	→ PERMANECEM
Documento fiscal emitido	→ INTOCADO

O titular precisa ser informado disso. A resposta ao pedido diz o que foi apagado e o que foi conservado, com o motivo. Prometer exclusão total e manter nota fiscal é pior que explicar a diferença.

E a anonimização é **irreversível e destrutiva no lugar**. Não se guarda um mapa de pedido para titular em outra tabela; isso seria pseudonimização disfarçada de anonimização, e o dado continuaria sendo pessoal.

15.4 A restrição que mais afeta o código

Nenhum dado pessoal existe apenas dentro de texto livre.

ADR-013 §6

Todo registro que contenha dado pessoal referencia um identificador estável do titular, indexado. Sem isso, exclusão vira varredura de texto em todas as tabelas de todos os serviços — lento, incompleto e impossível de auditar.

Consequência prática: telefone dentro do corpo de uma mensagem de conversa é inevitável, e por isso **o conteúdo da conversa é excluído, não anonimizado**. Não se tenta limpar texto livre.

15.5 Retenção

O QUÊ	PRAZO	DEPOIS
Pedido, itens, liquidação, ajustes	5 anos	anonimização do titular; valores permanecem
Fechamento de expediente	5 anos	anonimização do entregador; valores permanecem
Conteúdo de mensagem	180 dias após o encerramento	exclusão, imposta por índice no banco
Áudio	descartado após a transcrição	—
Contato: telefone, nome, endereços	enquanto ativo + 24 meses	exclusão
Log de aplicação	30 dias	descarte — é diagnóstico, não histórico
Backup	30 dias rolando	sobrescrito

Os cinco anos **não se aplicam ao pedido inteiro**. Aplicam-se ao que precisa ser reconstruído: valores, itens, método liquidado, quem autorizou. O nome e o telefone do cliente não são necessários para provar quanto foi vendido — e é por isso que a anonimização funciona.

O PONTO QUE TODO MUNDO ESQUECE

A exclusão vale para o dado vivo. **Backup restaurado ressuscita o que foi apagado**. Por isso os backups rolam em trinta dias e o procedimento de restauração inclui reaplicar as exclusões registradas com data posterior à do backup — é passo obrigatório do runbook, não observação.

No MVP a exclusão não é automatizada: é um **procedimento operacional escrito**, executado à mão contra cada banco. Automatizar antes de existir demanda é construir uma saga para zero pedidos por ano. O que **não** pode esperar é o alcance por identificador — sem ele, nem o procedimento manual funciona.

As vinte e uma decisões

Uma ADR responde *por que* uma decisão foi tomada, com alternativas rejeitadas e consequências negativas assumidas. Uma ADR sem "alternativas consideradas" e sem consequências negativas é decoração, não registro de decisão.

#	DECISÃO	ESTADO
001	Monorepo para os oito serviços, o gateway e os contratos	aceita
002	Um banco por serviço, sem exceção	aceita
003	Mensageria híbrida com MQTT	sem objeto
004	Um pedido pertence a exatamente um estabelecimento	aceita
005	PostGIS e geoconsulta	sem objeto
006	Carrinho: durabilidade, validade e ordem do checkout	a escrever — depende do marco 3
007	Mongock para versionamento de esquema no MongoDB	aceita
008	MongoDB como replica set de nó único	aceita
009	Modelo de valores do pedido	aceita · emendada
010	Saga do pedido: pivô em PRONTO, pagamento fora da transação	aceita · reescrita
011	Autorização comercial: cache em processo, invalidação por evento, fail-closed	aceita
012	Roteamento do gateway por recurso, não por serviço	aceita
013	Retenção, anonimização e exclusão de dados pessoais	aceita
014	Não adotar banco em memória; Testcontainers como fonte de verdade	aceita · emendada
015	Emitir JWT sem servidor de autorização completo	aceita
016	Front-end mínimo antes da PWA completa	aceita
017	MongoDB como decisão de aprendizado	aceita
018	Snapshot de opções no item e cotação pelo catálogo	aceita
019	Cotação de entrega por distância geodésica	revogada — ver 020
020	Taxa de entrega por área nomeada	aceita
021	Catálogo de serviços do MVP — oito serviços	aceita
022	A remuneração do entregador pertence ao vínculo	aceita
023	Fronteira order × payment: o pedido é dono do registro	aceita
024	Desconto de retirada, não preço por modalidade	aceita

16.1 As quatro que sustentam as outras

Com vinte e uma escritas, "leia as ADRs" deixou de ser instrução útil. Quem chega no projeto lê estas primeiro, nesta ordem:

DECISÃO	POR QUÊ
004 — um estabelecimento por pedido	é a premissa de que cinco outras dependem
002 — banco por serviço	explica de onde vêm saga, outbox e duplicação
021 — catálogo de serviços	diz quais serviços existem e por que dois saíram
009 — modelo de valores	o dinheiro, que é o que o produto existe para acertar

16.2 Duas decisões sobre as decisões

Emenda, não reescrita. Uma ADR alterada por revisão de escopo mantém o número e ganha um bloco de emenda com data e motivo. O texto abaixo do bloco é sempre o vigente — a versão anterior vive no histórico do Git, que é onde histórico deve viver.

EMENDA NÃO TERMINA NO GREP

Varredura por padrão encontra texto que contradiz a decisão nova. Não encontra o que está **ausente** — uma ressalva que falta, um campo com nome que virou enganoso, um documento que deveria citar a decisão e não cita. Depois do grep, leem-se as seções que tocam o mesmo dado que a decisão governa. Foi assim que se descobriu um campo com o nome da base legal que a ADR-013 havia rejeitado, e o congelamento do endereço sem a ressalva que torna a anonimização possível.

Marcos de entrega

Cada marco entrega valor operável, não uma camada técnica.

MARCO	ENTREGA	O COMERCIANTE CONSEGUE
0 ✓	Esqueleto: build, módulos, Compose, CI	—
1	Conta, estabelecimento, equipe e permissões	cadastrar a loja e o time, com poderes diferentes
2	Cardápio com opções e disponibilidade qualitativa	publicar o cardápio e marcar o que acabou
3	Painel de pedidos e ciclo de preparo	receber e acompanhar pedidos numa tela
4	Liquidação presencial, troco e Pix confirmado	registrar como cada pedido foi pago de verdade
5	Entregadores, vínculo, remuneração e despacho	despachar e saber quem está com cada pedido
6	Fechamento de expediente	fechar o dia com extrato por entregador
7	Canal WhatsApp determinístico	receber pedido pelo WhatsApp com menu numerado
8	Pagamento online e emissão fiscal	cobrar antecipado e emitir documento
9	Atendimento assistido e escalonamento	atender sozinho a maior parte dos pedidos
10	Controle quantitativo e substituição de item	operar minimercado com contagem de estoque
11	Rastreamento e telemetria	mostrar a entrega no mapa

Ao fim do marco 6 existe produto completo e vendável.

PRD §10.1

A loja cadastra equipe e cardápio, recebe e prepara pedidos, despacha com entregador próprio, registra como cada pedido foi liquidado e fecha o expediente com extrato. Nenhum marco posterior é necessário para que isso funcione, e a dor que decide a compra já está resolvida.

Os marcos 7 a 9 ampliam o alcance — trazem o pedido para dentro do canal onde ele já nasce. Os marcos 10 e 11 abrem segmentos novos.

17.1 O que o marco 1 precisa entregar além do óbvio

Três requisitos técnicos foram criados por decisões e não existem ainda. Os três são do marco 1, e nenhum deles aparece na descrição do marco:

- o filtro de segurança do gateway que libera **exatamente** o prefixo de webhooks e exige autenticação em todo o resto;
- a verificação no build que falha se um módulo de serviço declarar outro como dependência;
- a confirmação, com o gateway no ar, de que as rotas foram de fato registradas — configuração de roteamento ignorada é silenciosa: o serviço sobe e simplesmente não roteia.

O que ainda não existe

Esta é a seção mais útil do documento. Um documento de arquitetura que só descreve o que funciona é propaganda — e, pior, faz o leitor descobrir os buracos por conta própria, geralmente sob pressão.

Nada aqui é surpresa: cada item já está nomeado em algum documento do repositório, quase sempre numa seção chamada "o que este documento deliberadamente não decide". O valor de listá-los juntos é que a soma tem um tamanho, e o tamanho é informação.

18.1 Decisões de produto sem dono

DECISÃO	PRAZO	POR QUE IMPORTA
Numeração sequencial de pedido por loja	marco 3	reinicia diariamente? por ano? nunca? Precisa ser decidido antes do primeiro pedido gravado
Fuso horário do estabelecimento	marco 1	todo o horário de funcionamento presume um fuso que não está modelado. Loja única em Recife funciona por acidente; a primeira em outro fuso quebra
Recuperação do administrador único	antes do primeiro cliente	a invariante garante que <i>existe</i> um administrador; não garante que alguém consegue entrar como ele. Telefone trocado, aparelho perdido
Controles comerciais contra abuso	antes do marco 3	teto para cliente sem cadastro, bloqueio por reincidência. São regra de estabelecimento e não têm dono
Jornada que cruza a meia-noite	antes do marco 6	o turno da pizzaria termina às 2h. A jornada é do turno, não do dia civil — mas "total do dia" precisa de um critério, e ele não foi decidido
Idioma da nomenclatura	antes do marco 2	domínio em português e infraestrutura em inglês é a sugestão; a ADR-018 usa inglês e precisa de um passe de tradução ou de uma decisão explícita de conviver com os dois
Prazo para o Pix pendente virar não liquidado	marco 4	depende do provedor; hoje fica pendente indefinidamente e aparece no fechamento
Pedido mínimo	sem prazo	não existe no modelo. Um desconto de retirada de R\$ 5 num pedido de R\$ 10 é legal, e nada impede

18.2 Coisas nomeadas, decididas e não construídas

O QUÊ	SITUAÇÃO
Fila morta e procedimento operacional	as duas pernas retriáveis da saga não têm compensação. "Tentar para sempre" sem um lugar onde a mensagem morre e alguém olha é como se perde pedido em silêncio. Nomeado na ADR-010 como consequência negativa; não existe
Fechamento de caixa da loja	o sistema apura "quanto o entregador tem a receber". Não apura "quanto entrou na loja no total", que incluiria retirada e venda de balcão. Nomeado, sem dono — e a ADR-024 aumentou o peso disso ao dar receita própria à retirada

O QUÊ	SITUAÇÃO
Reconciliação entre pedido e pagamento	identificador e valor existem nos dois serviços. Divergência — "o provedor confirmou e o pedido continua pendente" — é possível, e precisa de procedimento, não de um catch. Pendência do marco 4
Saga de exclusão de titular	desenho definido na ADR-013; no MVP é procedimento manual auditado. Vira dívida com prazo no momento em que houver cliente real
Governança de versionamento de eventos	o esquema tem envelope e regra de nova versão. Falta definir o que conta como mudança compatível e como se sabe que nenhum consumidor depende mais da anterior
Documento de domínio de pagamento e estorno	existem seis documentos de domínio e nenhum de pagamento. Há decisão sobre a fronteira e sobre a saga, mas ninguém escreveu a máquina de estados do pagamento — e estorno é a parte menos desenhada do sistema
ADR-006 — carrinho	a única decisão sem escrever. Depende do desenho do carrinho, que só chega no marco 3; escrevê-la agora seria decidir no vazio

18.3 Requisitos não funcionais que não estão escritos

SLOs mensuráveis, matriz técnica de falha e degradação, objetivos de ponto e de tempo de recuperação, e testes de resiliência a falhas distribuídas. **Nenhum dos quatro existe.**

Os quatro compartilham uma propriedade: são sobre **operar** o sistema, e nada opera ainda. Escrever "disponibilidade de 99,9%" para um sistema com zero usuários e nenhum ambiente de produção produz um número que ninguém mede e que envelhece sozinho no documento.

Com uma exceção que vale aceitar agora: **a matriz de falhas tem metade de design.** "O que acontece quando o provedor de pagamento não responde" e "o que acontece quando o canal cai" mudam o código, não só o runbook. Essa metade entra; a metade de recuperação operacional espera haver operação.

Sobre backup especificamente: ele aparece hoje num único lugar, e por um motivo curioso — o procedimento de exclusão manda reaplicar as exclusões posteriores à data do backup, senão restaurar ressuscita dado que o titular pediu para apagar. É o único ponto onde backup foi pensado, e é pouco.

18.4 Divergências entre documentos, encontradas na revisão desta versão

Quatro, todas pequenas, todas do mesmo tipo: uma decisão aplicada em parte, ou uma nota que envelheceu depois que o problema que ela descrevia foi corrigido.

ONDE	O QUÊ
Evento de expediente	a renomeação foi decidida e está meio aplicada : o conversation já consome o nome novo, o merchant ainda publica o antigo, e o catalog ainda consome o antigo. Hoje o conversation escuta um evento que ninguém publica
Esquema do envelope de evento	traz como exemplos nomes em inglês da v1.0, enquanto todos os eventos do sistema são nomeados em português com sufixo de versão
Notas sobre o PRD em dois documentos de domínio	dizem que anotações do PRD estão desatualizadas quanto a marcos. O PRD já foi corrigido — a nota de divergência é que ficou velha

ONDE

Diagramas C4 em
docs/architecture/

O QUÊ

dois arquivos-marcador dizendo "a escrever na Fase 0", com a nomenclatura de marcos anterior. Este documento é o que estava faltando ali

18.5 Pendência jurídica

Os prazos de retenção da seção 15 precisam de revisão por advogado antes do primeiro cliente real. Faltam também a nomeação de encarregado e o canal de atendimento ao titular, o texto da política de privacidade, e o teste de balanceamento do legítimo interesse — que precisa ser escrito e guardado, não só citado.

**Um sistema honesto sobre o que não tem
é mais confiável que um sistema silencioso sobre isso.**

Glossário

TERMO	SIGNIFICADO
Estabelecimento	a loja. Unidade de isolamento de dados, equipe e cardápio
Vínculo	relação entre uma pessoa e um estabelecimento, com papel e permissões próprios. É onde a permissão mora
Modalidade	entrega ou retirada no balcão. São duas, e só duas
Método declarado	o que o cliente disse que pagaria
Método liquidado	o que de fato aconteceu. Divergir do declarado é o caso normal
Liquidação	registro, dentro do pedido, de que o dinheiro foi recebido: método, valor efetivo e responsável pela custódia
Custódia	responsabilidade por valor recebido fora da plataforma — em regra, o entregador
Jornada	turno de um entregador num estabelecimento, com abertura, movimento e fechamento
Lançamento	item somente-inserção dentro da jornada. Não confundir com liquidação: toda entrega produz os dois, toda retirada produz só a liquidação
Divergência	diferença entre o dinheiro esperado e o apurado no fechamento. Nunca é compensada no saldo
Disponibilidade qualitativa	estado do produto — disponível, acabando, esgotado hoje — sem contagem de saldo
Área de entrega	bairro ou faixa de CEP atendido, com taxa própria. Taxa zero é válida e distinta de área não atendida
Ajuste	lançamento que corrige um valor sem apagar o original
Pivô	o ponto da saga a partir do qual não se desfaz mais, só se avança. Aqui é PRONTO
Congelamento	cópia, no momento do pedido, de tudo que afeta valor. Depois dele o pedido não consulta mais catálogo nem cadastro
Outbox	tabela onde o evento é gravado na mesma transação do dado de negócio, para que ou os dois aconteçam ou nenhum
Fail-closed	negar quando o serviço de autorização não responde. Assumido com o custo declarado

Documento gerado em 24 de agosto de 2026 a partir do estado do repositório naquela data: PRD, seis documentos de domínio, vinte e uma decisões aceitas, um runbook e os contratos. Onde este documento e o repositório divergirem, o repositório está certo.